



SW개발/HW제작 설계서

프로젝트 명 : 독거노인의 식생활 관리를 위한 AI 기반 스마트 냉장고

2026. 07. 11

(팀명) 프로젝트팀 * 팀원 이름 및 소속 본문 내 기입 불가

수행 단계별 주요 산출물

단계	산출물	적용	수록
환경분석	시장/기술환경분석서, 설문조사결과서, 인터뷰결과서	선택 포함	3-6p
요구사항분석	요구사항정의서, 유즈케이스정의서	필수/선택 포함	7-9p
아키텍처설계	서비스구성도, 서비스흐름도, UI/UX 정의서, 하드웨어/센서구성도	필수 포함	10-14p
기능설계	메뉴구성도, 화면설계서, ERD, 기능흐름도, 알고리즘명세서	필수 포함	15-25p
개발/구현	하드웨어설계도, 프로그램목록, 테이블정의서, 핵심소스코드	필수 포함	26-30p
참조	개발환경, S/W/H/W 기능사진, 동영상 콘티, 프로젝트관리	참조 포함	31-35p

작성 기준

원본 양식의 샘플 내용은 모두 프로젝트 내용으로 대체하였다. 개인정보 보호 지침에 따라 팀원 이름과 소속은 본문에 기입하지 않았다.

| 시장/기술 동향 분석

사회 환경

- 고령화와 1인 가구 증가로 혼자 식사를 준비하는 사용자의 냉장고 재고 관리 부담이 커지고 있다.
- 독거노인은 보유 식재료를 기억하기 어렵고, 방치된 식재료는 음식물 폐기나 영양 불균형으로 이어질 수 있다.
- 복지 현장에서는 생활 밀착형 데이터를 활용한 돌봄 보조 서비스의 수요가 증가하고 있다.

사용자 문제

- 식재료를 직접 입력하고 수정하는 기존 앱 방식은 고령층에게 번거롭다.
- 냉장고 안의 실제 재료와 앱 재고 목록이 쉽게 불일치한다.
- 무엇을 만들 수 있는지 바로 알기 어렵고, 부족한 재료도 따로 확인해야 한다.

서비스 기회

- 카메라 촬영과 AI 인식을 결합하면 입력 부담을 줄일 수 있다.
- 재고 데이터와 레시피 데이터를 연결하면 식재료 활용도를 높일 수 있다.
- 스마트홈/돌봄 플랫폼과 연계 가능한 생활지원형 서비스로 확장할 수 있다.

분석 항목	현재 방식의 한계	본 프로젝트 적용 방향
재고 입력	사용자가 식재료명을 직접 입력해야 하며 누락이 잦음	문 열림 이벤트와 카메라 촬영으로 자동 등록 흐름 구현
식재료 확인	냉장고를 열어 직접 확인해야 하며 기억 의존도가 큼	앱에서 냉장고별 보유 재료, 수량, 인식 이미지를 확인
식사 준비	보유 재료와 레시피를 사용자가 따로 대조해야 함	보유 재료 기반 추천과 부족 재료 안내 제공
돌봄 확장	재고 상태가 데이터화되지 않아 외부 서비스 연계가 어려움	API/DB 기반으로 향후 보호자, 복지기관 연계 가능

| 시장/기술 동향 분석

기술 영역	동향	본 프로젝트 적용
AI 이미지 인식	경량 객체탐지 및 분류 모델을 활용해 edge device 또는 소형 서버에서 실시간 추론을 수행하는 방향으로 발전	YOLOv8n detector와 식재료 분류 모델(best.pt)을 조합하여 후보 영역 탐지 및 class 추론 수행
AIoT	센서, 카메라, 네트워크, AI가 결합되어 일상 환경 데이터를 자동 수집하고 서비스 판단에 활용	Raspberry Pi 5, 카메라, 리드스위치, Flask API를 연결하여 문 열림/닫힘 이벤트 기반으로 동작
모바일 서비스	사용자는 자동 수집된 정보를 앱에서 확인하고 예외 상황만 수정하는 방식 선호	Flutter 앱에서 재고 목록, 상세 수정, 레시피 추천, 냉장고 선택 기능 제공
데이터 관리	이미지 원본, crop, confidence, 사용자 수정 이력을 함께 관리해야 모델 개선이 가능	MySQL에 재고와 레시피를 저장하고 업로드 이미지 경로와 confidence를 함께 보관

차별화 요약

- 단순 앱 입력이 아니라 냉장고 사용 행위 자체를 인식 흐름으로 전환한다.
- YOLO 탐지, contour 후보, center fallback을 조합해 배치 변화에 대응한다.
- 재료 등록에서 끝나지 않고 레시피 추천과 부족 재료 안내까지 연결한다.

참고 근거

- 고령화와 1인 가구 증가에 따른 생활지원 서비스 필요성
- AI와 IoT를 결합하는 AIoT 기술 흐름
- 스마트홈, 돌봄, 식생활 관리 서비스의 데이터 연계 가능성

| 설문조사 분석

구분	설문 항목	분석 결과	설계 반영
대상	독거노인, 1인 가구, 보호자 관점의 식재료 관리 불편 여부	보유 재료 확인과 소비 계획 수립에 어려움이 있다고 가정	앱 첫 화면에서 재고 확인과 레시피 확인을 바로 선택
입력 부담	식재료를 직접 앱에 등록하는 방식의 편의성	직접 입력은 지속 사용을 방해하는 핵심 요인	카메라 자동 인식 후 사용자는 오인식만 수정
정보 표시	필요한 재고 정보	이름, 수량, 상태, 촬영 이미지, 등록 시점이 필요	InventoryItem 모델에 이미지 URL, confidence, note 포함
추천	보유 재료 기반 조리 도움 필요성	현재 재료로 만들 수 있는 음식과 부족 재료 안내 필요	recipes API에서 matched/missing count 산출
접근성	복잡한 설정과 화면 이동의 부담	핵심 기능을 큰 카드와 명확한 메뉴로 제공해야 함	홈 화면을 재고 확인/레시피 확인 2개 카드 중심으로 구성

설문 기반 핵심 요구

자동 등록, 쉬운 확인, 직접 수정, 부족 재료 안내, 냉장고별 분리 관리가 핵심 요구로 정리된다. 설문은 실제 사용자 테스트 전 기획 가설을 정리한 분석 자료이며, 향후 고령층 대상 사용성 테스트로 보완한다.

| 인터뷰 결과서

가설 대상	예상 의견	문제 해석	설계 반영
독거노인 사용자	냉장고에 무엇이 있는지 자주 잊고, 장보기 전 확인이 어렵다.	기억 의존형 재고 관리는 실패 가능성이 높다.	냉장고 앱에서 현재 재고를 바로 확인
보호자/돌봄 담당자	식사를 잘 챙기는지 간접적으로 알 수 있는 단서가 필요하다.	재고 변화 데이터는 돌봄 보조 정보가 될 수 있다.	향후 알림/공유 기능을 확장 가능한 API 구조로 설계
개발자	카메라 위치와 조명, 식재료 배치가 일정하지 않아 인식을 저하가 우려된다.	고정 crop만으로는 실제 환경 대응이 어렵다.	YOLO box + contour proposal + fallback 구조 적용
서비스 운영 관점	오인식 결과가 그대로 저장되면 신뢰도가 떨어진다.	자동화와 사용자 검수의 균형이 필요하다.	UNRECOGNIZED 상태, 직접 수정/삭제 기능 제공

1. 관찰

냉장고 확인과 식사 준비가 분리됨

2. 문제정의

입력 부담과 기억 의존도를 낮춰야 함

3. 해결방향

촬영, 인식, 저장, 추천을 자동 연결

4. 검증

사용자 수정 기능으로 오인식 보완

※ 실제 인터뷰 결과가 아닌 기획 단계의 가설이며, 목표 사용자 검증 후 갱신한다.

| 요구사항 정의서

ID	구분	요구사항명	상세 설명	우선순위
REQ-SW-01	S/W	회원가입/로그인	사용자는 이름, 이메일, 비밀번호로 계정을 생성하고 로그인할 수 있다.	상
REQ-SW-02	S/W	냉장고 관리	사용자는 여러 냉장고를 등록, 선택, 삭제하고 현재 냉장고를 지정할 수 있다.	상
REQ-SW-03	S/W	재고 조회/수정	냉장고별 식재료 목록을 조회하고 수량, 단위, 상태, 메모를 수정할 수 있다.	상
REQ-SW-04	S/W	레시피 추천	보유 식재료와 레시피 필요 재료를 비교하여 추천 순서와 부족 재료를 제공한다.	중
REQ-HW-01	H/W	문 열림 감지	리드스위치를 통해 냉장고 문 열림/닫힘 이벤트를 감지한다.	상
REQ-HW-02	H/W	카메라 촬영	문 이벤트 또는 연속 모드에서 냉장고 내부 이미지를 촬영한다.	상
REQ-AI-01	AI	식재료 인식	이미지에서 후보 영역을 추출하고 식재료명과 confidence를 산출한다.	상
REQ-API-01	API	자동 업로드/소비	인식 결과를 /upload로 등록하고 필요 시 /consume으로 수량을 차감한다.	상

| 요구사항 정의서

구분	기능	설명
S/W	식재료 이미지 인식 결과 저장	Raspberry Pi에서 전달한 label, confidence, 원본 이미지, crop 이미지를 서버 DB와 업로드 폴더에 저장한다.
S/W	재고 목록 관리	냉장고별 재고를 조회하고 사용자가 직접 추가, 수정, 삭제할 수 있도록 한다.
S/W	레시피 추천	현재 보유 재료와 레시피 필요 재료를 비교해 missing_count가 낮은 순서로 제공한다.
S/W	오인식 검수	인식되지 않은 항목은 UNRECOGNIZED 상태로 표시하고 사용자가 이름과 상태를 수정할 수 있게 한다.
H/W	리드스위치 문 이벤트	GPIO17 입력을 기준으로 문 열림/닫힘을 판단하고 scan workflow를 실행한다.
H/W	카메라 프레임 수집	Picamera2 또는 OpenCV camera backend로 640x480, 30FPS 프레임을 수집한다.
H/W	미리보기 스트림	MJPEG preview stream을 제공하여 장착 위치와 인식 박스를 브라우저에서 확인한다.
AI	후보 영역 추출	YOLO detector box, OpenCV contour proposal, center fallback 순으로 crop 후보를 확보한다.
AI	안정 프레임 판정	동일 label signature가 stable_frames 이상 반복될 때만 업로드하여 중복과 오탐을 줄인다.

| 유즈케이스 정의서

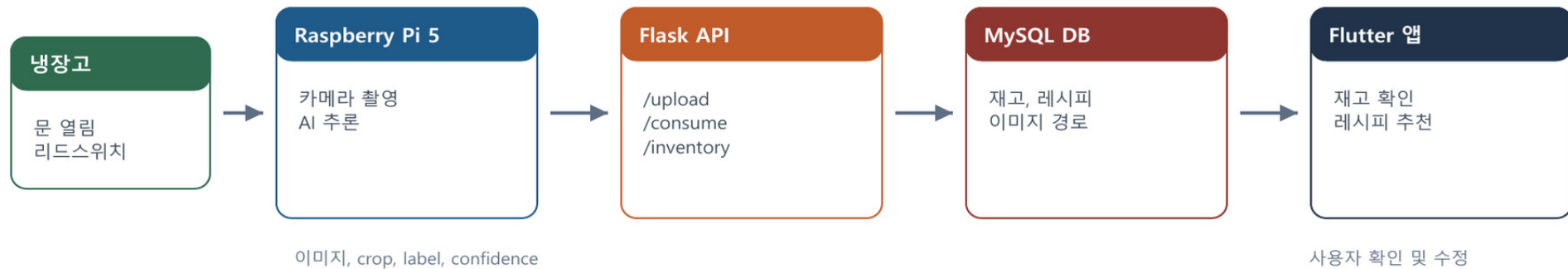
Actor	Use Case	목표
사용자	회원가입/로그인	개인 냉장고 데이터를 구분하여 사용
사용자	냉장고 선택	관리할 냉장고를 선택
사용자	재고 확인	현재 보유 식재료와 상태 확인
사용자	재고 수정	오인식, 수량, 메모를 수정
사용자	레시피 확인	보유 재료로 가능한 조리 정보 확인
Raspberry Pi	촬영/인식/업로드	문 이벤트 기반으로 재료 자동 등록
백엔드	데이터 저장/추천	재고와 레시피를 통합 관리



대표 유즈케이스: 식재료 자동 등록

- 사전조건: 사용자는 냉장고를 등록했고 백엔드와 Raspberry Pi가 같은 네트워크에서 동작한다.
- 기본흐름: 문 열림 감지 -> 카메라 촬영 -> 후보 crop 추출 -> AI 분류 -> 안정 프레임 확인 -> /upload 전송 -> 앱 재고 목록 갱신
- 예외흐름: confidence가 낮거나 미등록 재료이면 UNRECOGNIZED로 저장하고 사용자가 앱에서 수정한다.

| 서비스 구성도 - 서비스 시나리오



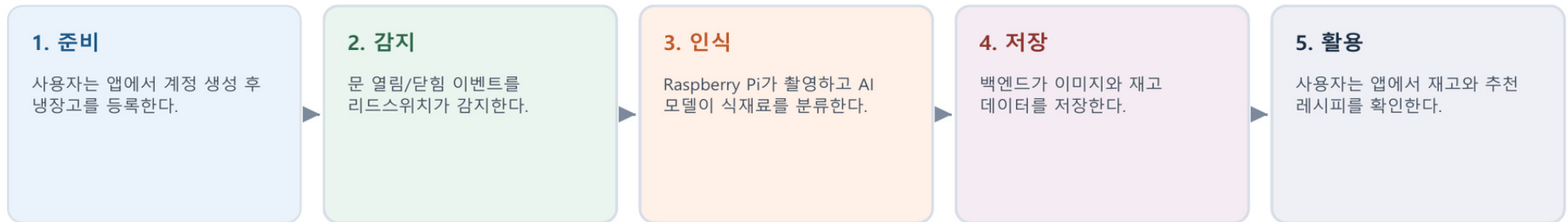
서비스 설명

- 냉장고 문이 열리면 Raspberry Pi가 카메라 프레임을 수집한다.
- AI 모듈은 이미지에서 식재료 후보 영역을 찾고 식재료명을 분류한다.
- Flask API는 인식 결과와 이미지를 저장하고 MySQL 재고 데이터로 반영한다.
- Flutter 앱은 재고 현황과 레시피 추천 결과를 사용자에게 제공한다.

데이터 항목

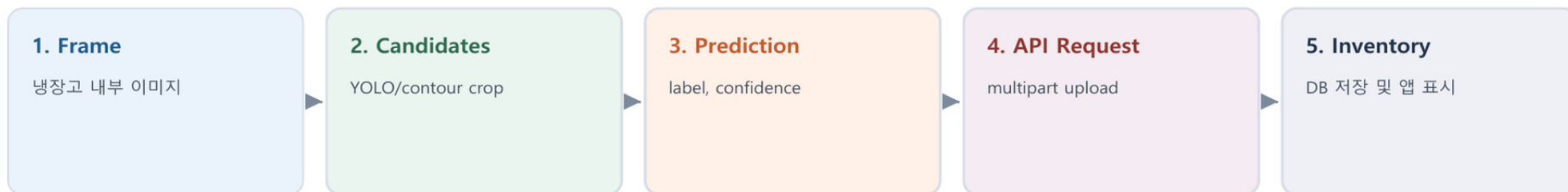
- 원본 이미지: 냉장고 내부 전체 프레임
- crop 이미지: 인식 후보 영역
- label/confidence: AI 판정 결과
- status: RECOGNIZED, UNRECOGNIZED, USER_CONFIRMED
- fridge_id: 냉장고별 재고 분리 기준

| 서비스 구성도 - 서비스 시나리오



시나리오	정상 흐름	예외 처리
신규 식재료 추가	문 열림 -> stable recognition -> /upload -> fridge_items insert	동일 식재료 반복 감지 시 cooldown으로 중복 업로드 방지
식재료 소비	문 닫힘 scan -> /consume -> 기존 재고 수량 차감	대상 재고가 없으면 consumed=false로 응답하고 DB 변경 없음
오인식 보완	UNRECOGNIZED 항목을 앱에서 선택 -> 이름/상태 수정	사용자 수정 후 USER_CONFIRMED 상태로 관리
레시피 추천	보유 재료명 set과 recipe_ingredients 비교 -> missing_count 계산	검색어가 있으면 recipe name LIKE 조건으로 필터링

| 서비스 흐름도



흐름	입력	처리	출력
카메라	문 이벤트 또는 continuous mode	frame read, preview update	BGR frame
후보 추출	frame	detector box, contour, center fallback	crop candidates
분류	crop image	YOLO classifier top5 probability 확인	Prediction(label, confidence)
안정화	prediction signature	stable_frames 횟수와 cooldown 확인	uploadable candidates
서버 저장	image, crop_image, label, fridge_id	ingredient resolve, status 결정	fridge_items row
앱 표시	REST API response	JSON decode, URL normalize	재고 카드, 상세 화면, 레시피 추천

| UI/UX 정의서 - 화면 설계서

UI 원칙

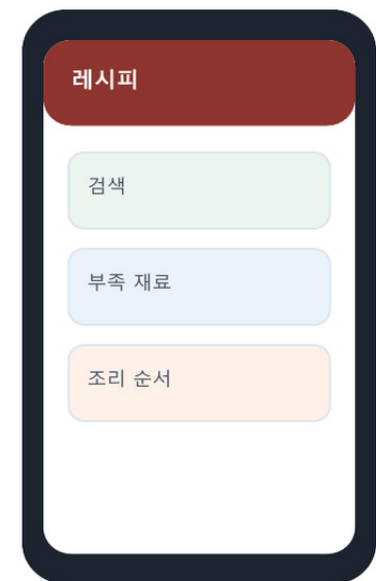
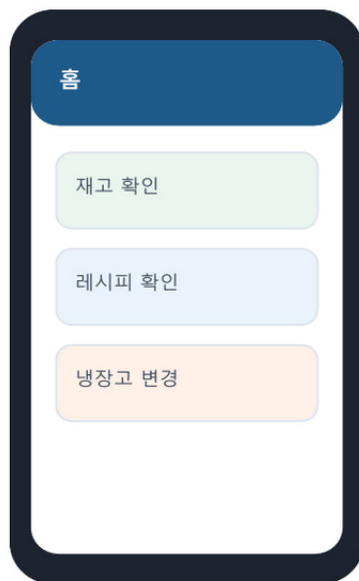
- 첫 화면은 로그인 이후 재고 확인과 레시피 확인으로 단순화한다.
- 재고 카드는 이미지, 이름, 수량, 상태를 한눈에 볼 수 있게 한다.
- 오인식 항목은 별도 상태 색상과 필터로 빠르게 찾게 한다.

접근성 원칙

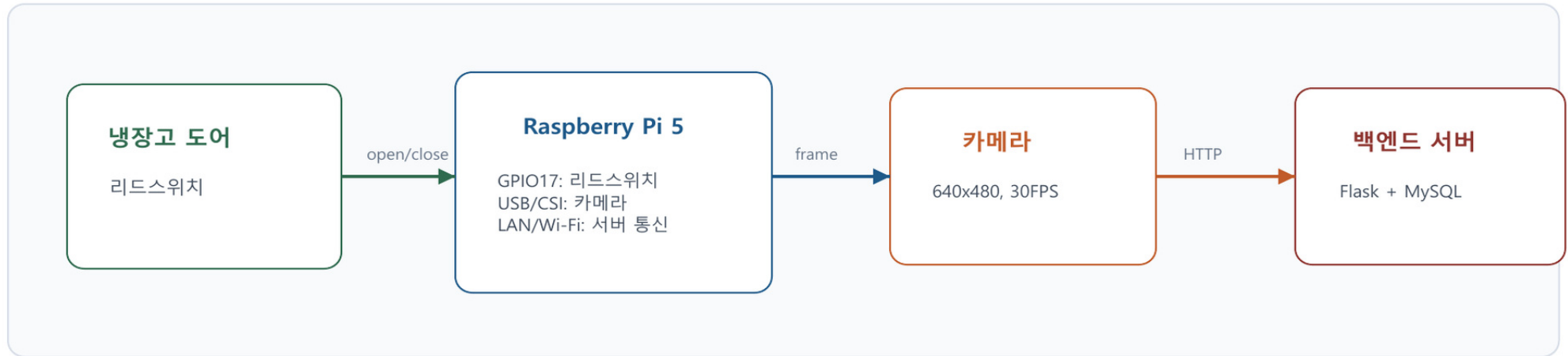
- 텍스트와 버튼 크기를 충분히 확보하고 복잡한 설정을 줄인다.
- 냉장고별 전환은 명확한 앱바 액션과 선택 화면으로 제공한다.
- 오류 발생 시 다시 시도 버튼과 짧은 메시지로 안내한다.

검수 원칙

- AI confidence와 원본/crop 이미지를 저장해 판정 근거를 남긴다.
- 사용자는 이름, 수량, 단위, 상태, 메모를 직접 수정할 수 있다.
- 사용자 수정은 USER_CONFIRMED 상태로 구분한다.



| 하드웨어/센서 구성도



부품/센서	연결	역할	비고
Raspberry Pi 5	LAN/Wi-Fi, CSI/USB camera, GPIO17	카메라 프레임 수집, AI 추론, 백엔드 업로드	Raspberry Pi OS 64-bit
카메라 모듈	CSI 또는 USB	냉장고 내부 이미지 촬영	기본 640x480, 30FPS
리드스위치	GND, GPIO17	냉장고 문 열림/닫힘 감지	문 열림 high 기준, 필요 시 low로 반전
백엔드 서버	HTTP REST	이미지 저장, 재고 DB 반영	Flask, MySQL
스마트폰/웹	HTTP API	재고 및 레시피 확인	Flutter app

| 메뉴 구성도



메뉴	하위 기능	관련 API
인증	로그인, 회원가입, 로그아웃	POST /auth/login, POST /auth/register
냉장고	목록 조회, 생성, 삭제, 현재 냉장고 선택	GET/POST/DELETE /fridges, PUT /users/{id}/current-fridge
재고	목록 조회, 상세 조회, 추가, 수정, 삭제	GET/POST/PUT/DELETE /inventory
레시피	추천 목록, 상세 조회, 검색	GET /recipes, GET /recipes/{id}

| 메뉴 구성도

시작 화면	조건	다음 화면	주요 액션
로그인	기존 사용자	홈 또는 냉장고 선택	로그인 성공 시 사용자의 current_fridge_id 확인
회원가입	신규 사용자	냉장고 선택	기본 냉장고 생성 및 계정 저장
냉장고 선택	등록 냉장고 없음	냉장고 추가	Floating button으로 냉장고 생성
홈	냉장고 선택 완료	재고 목록 또는 레시피 목록	큰 카드 2개로 핵심 기능 진입
재고 목록	카드 선택	재고 상세	수량, 단위, 상태, 메모 수정
레시피 목록	레시피 선택	레시피 상세	필요 재료, 부족 재료, 조리 순서 확인

메뉴 설계 특징

- 인증, 냉장고 선택, 홈, 재고, 레시피의 5개 큰 흐름으로 단순화한다.
- 자동 인식 기능은 사용자가 별도 메뉴를 조작하지 않아도 백그라운드에서 재고 목록으로 반영된다.
- 사용자는 앱에서 확인과 수정에 집중하며, 촬영과 인식은 H/W bridge가 담당한다.

| 화면 설계서

로그인

이메일

비밀번호

로그인

회원가입 이동

회원가입

이름

이메일

비밀번호

가입 완료

홈

오늘 냉장고

재고 확인

레시피 확인

로그아웃

기능번호	화면명	기능설명	처리내용
LOG-01	로그인	기존 사용자가 이메일과 비밀번호로 접속한다.	성공 시 사용자 정보와 냉장고 목록을 받아 홈으로 이동
REG-01	회원가입	신규 사용자가 계정 정보를 등록한다.	비밀번호 hash 저장, 기본 냉장고 생성
HOME-01	홈	선택된 냉장고 기준으로 핵심 기능을 제공한다.	재고 확인, 레시피 확인, 냉장고 변경, 로그아웃

| 화면 설계서



기능번호	화면명	입력 데이터	출력 데이터	예외사항
FRG-01	냉장고 선택	user_id, fridge_name	냉장고 목록	냉장고가 없으면 추가 안내
INV-01	재고 목록	fridge_id, 검색어, 필터	재고 카드 목록	서버 오류 시 retry 제공
INV-02	재고 상세	이름, 수량, 단위, 상태, 메모	수정된 재고 정보	잘못된 수량 입력 시 validation
RCP-01	레시피 목록/상세	fridge_id, query, recipe_id	추천 순서, 부족 재료, 조리법	검색 결과 없을 때 빈 상태 표시

| 화면 설계서 - 사용자 인터페이스(sw)



재고 카드에 표시되는 식재료 이미지 예시

재고 목록 화면 구성

- 상단: 냉장고명, 전체/정상/확인 필요 통계
- 검색: 식재료명, AI 추정명, 메모 검색
- 필터: 전체, 확인 필요, 정상
- 카드: 이미지, 상태, 이름, 수량, confidence

상태 표시 정책

- RECOGNIZED: AI 인식 완료
- UNRECOGNIZED: 사용자 확인 필요
- USER_CONFIRMED: 사용자 수정 완료
- 오인식은 상세 화면에서 이름과 상태 수정

항목	UI 컴포넌트	데이터 소스	설명
이미지	카드 상단 thumbnail	crop_image_url 또는 image_url	인식된 crop 우선 표시, 없으면 샘플 asset 표시
상태	badge/chip	status	확인 필요 항목을 색상으로 구분
수량	텍스트 + 단위	quantity, unit	정수는 소수점 없이 표시
검색	TextField	display_name, detected_name, note	사용자가 원하는 재료를 빠르게 찾을 수 있음

| 화면 설계서 - 사용자 인터페이스(sw)

레시피 목록

검색어 입력

오를렛

부족 재료 0

조리 10분

레시피 상세

필요 재료

부족 재료

조리 순서

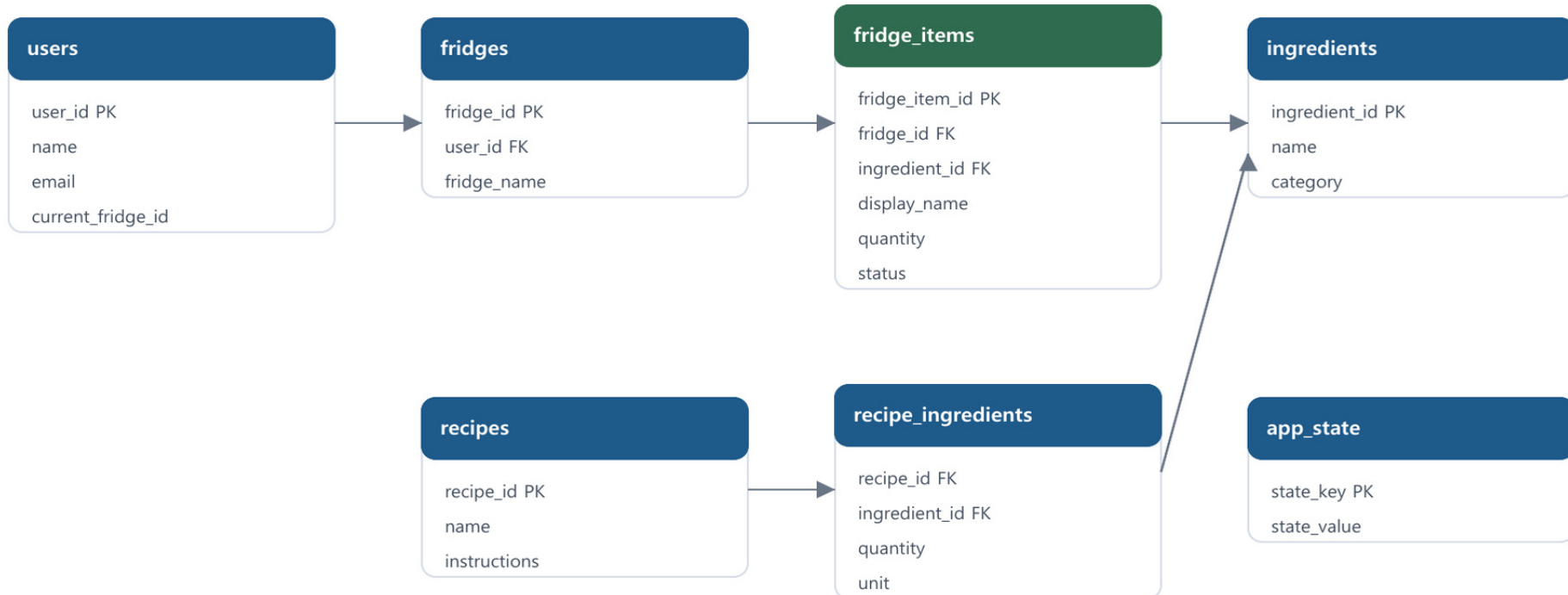
난이도

추천 기준

- 현재 냉장고의 보유 재료명 set을 조회한다.
- 레시피별 필요 재료와 비교해 matched_count, missing_count를 계산한다.
- missing_count가 낮고 matched_count가 높은 레시피를 우선 표시한다.
- 부족 재료를 함께 표시해 장보기 계획에 활용할 수 있게 한다.

출력 항목	설명	사용자 가치
레시피명/설명	간단한 요리명과 설명	무엇을 만들 수 있는지 빠르게 파악
조리시간/난이도	cooking_time, difficulty	사용 가능한 시간에 맞게 선택
부족 재료	missing_ingredients	현재 냉장고에 없는 재료 확인
조리 순서	instructions	선택 후 바로 조리 진행

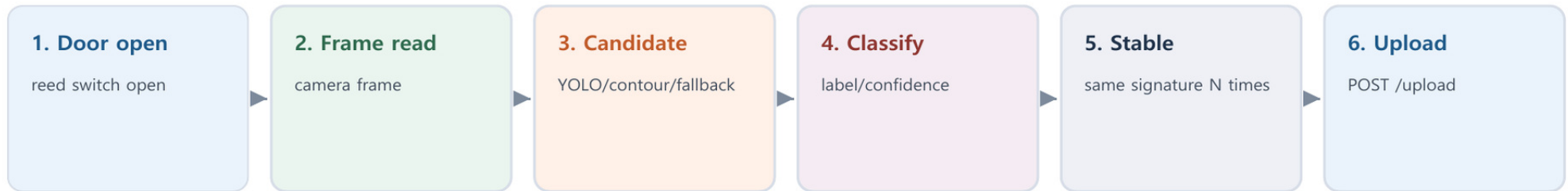
| 엔티티 관계도 - ERD



ERD 설명

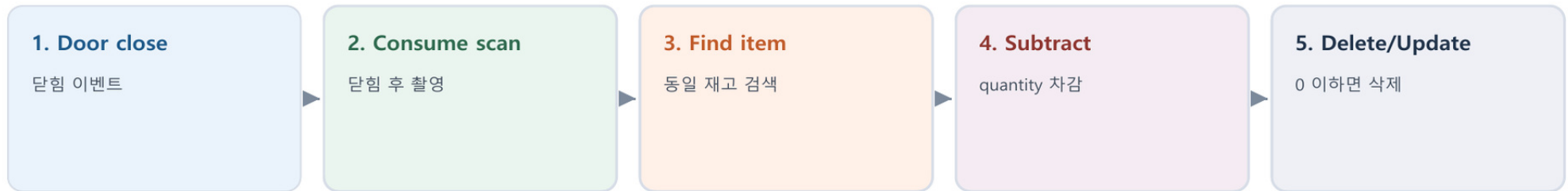
users는 여러 fridges를 소유하고, fridges는 fridge_items를 가진다. fridge_items는 ingredients와 선택적으로 연결되며, recipes와 ingredients는 recipe_ingredients로 N:M 관계를 구성한다. app_state는 현재 활성 냉장고 등 전역 상태를 저장한다.

| 기능 처리도(기능 흐름도)



단계	프로그램/모듈	처리 내용	실패/예외 처리
1	ReedDoorSensor	GPIO17 상태를 읽어 문 열림/닫힘을 판단	open_level high/low 옵션으로 반전 가능
2	Camera backend	Picamera2 우선, 실패 시 OpenCV camera 사용	카메라 미가용 시 오류 출력
3	collect_crop_candidates	detector, contour, center fallback 후보 수집	후보가 없으면 중앙 crop 사용
4	choose_prediction	분류 결과 top5와 confidence 기준으로 label 선택	min_confidence 미만이면 none 처리
5	scan_until_action	동일 signature가 stable_frames 이상 반복될 때 처리	scan_timeout 초과 시 중단
6	upload_prediction	이미지와 crop을 multipart로 /upload 전송	백엔드 미가용 시 사전 health check

| 기능 처리도(기능 흐름도)



흐름	조건	서버 처리	결과
add-on-open	문 열림 시 안정 인식	fridge_items INSERT	새 재료 자동 등록
consume-on-close	문 닫힘 시 안정 인식	가장 최근 동일 항목 quantity 차감	재고 감소 또는 삭제
not_found	소비할 동일 재고가 없음	DB 변경 없음, consumed=false 반환	앱 재고 유지
unrecognized	ingredient table에 없는 label	status=UNRECOGNIZED로 저장	사용자 확인 대상 표시
duplicate	동일 label 반복 감지	cooldown과 signature 비교	중복 등록 방지

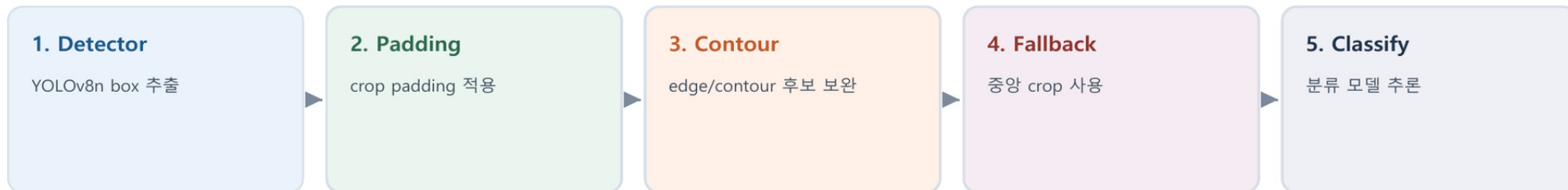
| 알고리즘 명세서

알고리즘	입력	처리	출력
후보 영역 추출	카메라 frame	YOLO detector box -> contour proposal -> center fallback 순으로 후보 crop을 생성하고 겹침이 큰 box는 제거	CropCandidate 목록
식재료 분류	crop image	trusted detector label은 직접 사용하고, 그 외는 classifier top probability와 min_confidence 기준으로 선택	Prediction(label, confidence)
안정 프레임 판정	분류 후보 목록	업로드 가능한 label signature가 동일하게 반복되는 횟수를 계산	stable upload/consume 여부
재고 등록	label, image, crop, fridge_id	ingredient alias 정규화 후 DB에 INSERT, ingredient 미확인 시 UNRECOGNIZED 처리	Inventory item
레시피 추천	냉장고 보유 재료, 레시피 필요 재료	레시피별 matched_count, missing_count 계산 후 정렬	추천 레시피 목록

판정 기준

min_confidence 기본값은 0.65, detector confidence 기본값은 0.30이며, stable_frames 기본값은 3이다. Raspberry Pi 성능에 따라 detection_imgsz, interval, crop_padding_ratio를 조정할 수 있다.

| 알고리즘 상세 설명서

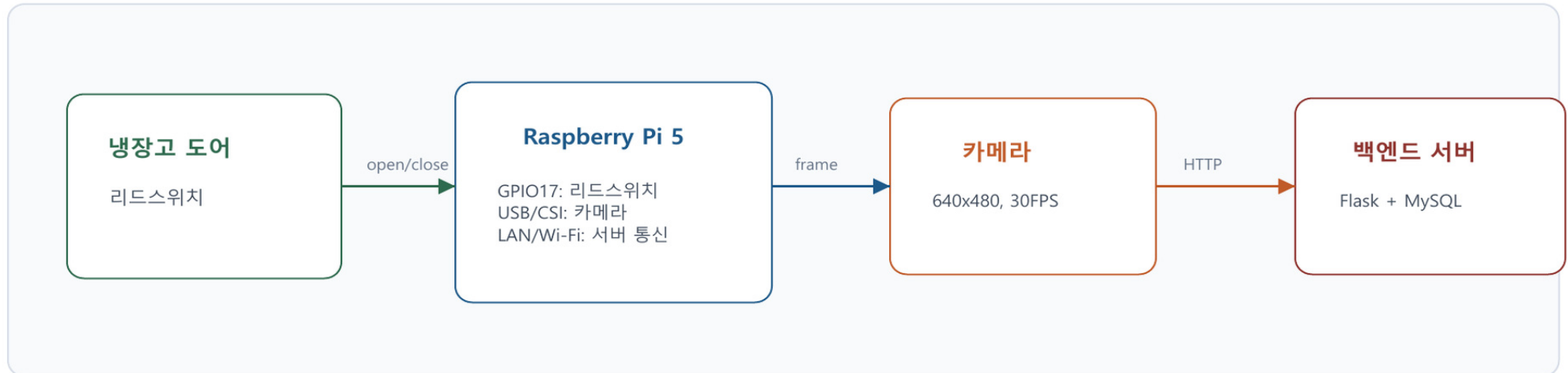


상세 항목	설명
box overlap 제거	기존 후보와 0.65 이상 겹치는 box는 중복 후보로 보고 제외한다.
crop padding	식재료 주변 맥락을 보존하기 위해 box 크기 비율 기반 padding을 적용한다.
trusted detector label	apple, banana, carrot 등 일반 detector가 신뢰할 수 있는 식품 label은 detector confidence를 그대로 사용한다.
background 처리	분류 top label이 background이고 confidence가 충분하지 않으면 다음 후보 label을 확인한다.
preview stream	인식 box와 상태 텍스트를 MJPEG stream으로 표시하여 장착 위치와 추론 결과를 확인한다.

효과

고정 중앙 crop만 사용하는 방식보다 냉장고 내부 배치 변화와 다양한 식재료 위치에 대응할 수 있으며, stable frame 판정과 cooldown으로 반복 업로드를 줄인다.

| 하드웨어 설계도



설계 요소	설계 내용	검증 방법
카메라 위치	냉장고 내부 상단 또는 문 안쪽에 설치하여 손이 들어오고 나가는 순간 모두 촬영 가능하도록 배치	preview stream으로 시야각 확인
리드스위치	문이 닫힐 때 자석과 가까워지고, 열릴 때 GPIO 상태가 변하도록 부착	once mode로 open/close 이벤트 테스트
전원	Raspberry Pi 5 안정 전원 사용	장시간 실행 시 재시작 여부 확인
네트워크	Pi와 백엔드가 같은 LAN 또는 Tailscale에서 통신	/health 및 /upload 응답 확인
서비스 실행	systemd service로 자동 실행 가능	journalctl 로그와 앱 재고 반영 확인

| 프로그램 - 목록

기능번호	분류	프로그램/파일	기능명	신규/기존
LOG-01	APP	login_screen.dart	로그인	신규
REG-01	APP	register_screen.dart	회원가입	신규
FRG-01	APP	fridge_selection_screen.dart	냉장고 등록/선택/삭제	신규
HOME-01	APP	home_screen.dart	홈 메뉴	신규
INV-01	APP/API	inventory_screen.dart, /inventory	재고 목록 조회	신규
INV-02	APP/API	inventory_detail_screen.dart, /inventory/{id}	재고 상세 수정/삭제	신규
RCP-01	APP/API	recipe_list_screen.dart, /recipes	레시피 추천 목록	신규
RCP-02	APP/API	recipe_detail_screen.dart, /recipes/{id}	레시피 상세	신규
AI-01	H/W/AI	pi_fridge_camera.py	촬영, 후보 추출, 분류, 업로드	신규
API-01	SERVER	app.py	인증, 냉장고, 재고, 레시피, 업로드 API	신규
DB-01	SERVER	init_db()	MySQL 테이블 생성 및 seed data	신규

| 테이블 정의서 - ERD

테이블	주요 컬럼	설명
users	user_id, name, email, password_hash, current_fridge_id	사용자 계정과 현재 선택 냉장고 관리
fridges	fridge_id, user_id, fridge_name, created_at	사용자별 냉장고 목록
ingredients	ingredient_id, name, category	AI label과 레시피 재료를 연결하는 기준 재료 마스터
fridge_items	fridge_item_id, fridge_id, ingredient_id, display_name, quantity, unit, status, image_path, crop_image_path, confidence	냉장고별 재고 및 AI 인식 결과 저장
recipes	recipe_id, name, description, instructions, cooking_time, difficulty	추천 레시피 본문
recipe_ingredients	recipe_id, ingredient_id, quantity, unit	레시피별 필요 재료 N:M 연결
app_state	state_key, state_value, updated_at	active_fridge_id 등 전역 상태 저장

상태값 정의

RECOGNIZED는 AI 인식 완료, UNRECOGNIZED는 사용자 확인 필요, USER_CONFIRMED는 사용자가 직접 입력하거나 수정한 항목을 의미한다.

| 핵심 소스코드(1)

backend/app.py - /upload

```
@app.route("/upload", methods=["POST"])
def upload_detection():
    image = request.files["image"]
    crop_image = request.files.get("crop_image")
    fridge_id = request.form.get("fridge_id", type=int)
    display_name = (request.form.get("label") or "").strip()
    confidence = request.form.get("confidence", type=float)

    image_path, crop_path = save_detection_files(image, crop_image)
    with get_connection() as conn:
        with conn.cursor() as cursor:
            fridge_id = resolve_target_fridge_id(cursor, fridge_id)
            ingredient = resolve_ingredient(cursor, display_name)
            status = "RECOGNIZED" if ingredient else "UNRECOGNIZED"
            cursor.execute("""INSERT INTO fridge_items
            (fridge_id, ingredient_id, display_name, quantity, unit, status,
            image_path, crop_image_path, confidence, detected_name,
            created_at, updated_at)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)""", ...)
            conn.commit()
    return jsonify(serialize_item(row)), 201
```

backend/app.py - /recipes

```
@app.route("/recipes", methods=["GET"])
def list_recipes():
    fridge_id = request.args.get("fridge_id", type=int)
    owned_names = load_owned_ingredient_names(fridge_id)
    recipe_rows = load_recipe_rows()
    results = []
    for recipe in recipe_rows:
        required_rows = load_required_ingredients(recipe["recipe_id"])
        missing = [row for row in required_rows if row["name"] not in owned_names]
        matched_count = len(required_rows) - len(missing)
        results.append({
            "recipe_id": recipe["recipe_id"],
            "name": recipe["name"],
            "matched_count": matched_count,
            "missing_count": len(missing),
            "missing_ingredients": missing,
        })
    results.sort(key=lambda item: (item["missing_count"], -item["matched_count"]))
    return jsonify(results)
```

| 핵심 소스코드(2)

backend/pi_fridge_camera.py

```
def collect_crop_candidates(frame, detector, settings):
    candidates = detector_crop_candidates(frame, detector, settings)
    for contour_candidate in contour_crop_candidates(frame, settings):
        append_non_overlapping(candidates, contour_candidate, settings.max_candidates)
    if not candidates:
        candidates.append(center_crop_candidate(frame, settings.crop_ratio))
    return candidates

def scan_until_action(camera, detector, classifier, args, action_url, action):
    last_signature = ()
    stable_count = 0
    while True:
        frame = camera.read()
        candidates = classify_frame(frame, detector, classifier, scan_settings(args))
        signature = prediction_signature(candidates)
        if signature and signature == last_signature:
            stable_count += 1
        else:
            last_signature = signature
            stable_count = 1 if signature else 0
        if signature and stable_count >= args.stable_frames:
            apply_candidates(action, action_url, args.fridge_id, frame, candidates, args.dry_run)
            return True
```

fridge_app/lib/services/api_service.dart

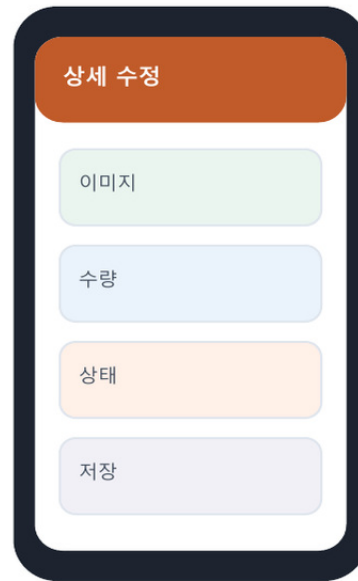
```
static Future<List<InventoryItem>> fetchInventory(int fridgeId) async {
    final response = await http.get(
        Uri.parse('$baseUrl/inventory?fridge_id=$fridgeId'),
    );
    final data = _decodeList(response);
    return data
        .map((item) => InventoryItem.fromJson(_normalizeItemUrls(item)))
        .toList();
}

static Future<List<RecipeSummary>> fetchRecipes({
    required int fridgeId,
    String query = '',
}) async {
    final response = await http.get(
        Uri.parse('$baseUrl/recipes?fridge_id=$fridgeId&q=${Uri.encodeQueryComponent(query)}'),
    );
    final data = _decodeList(response);
    return data.map((item) => RecipeSummary.fromJson(item)).toList();
}
```

| 참조-개발 환경 및 설명

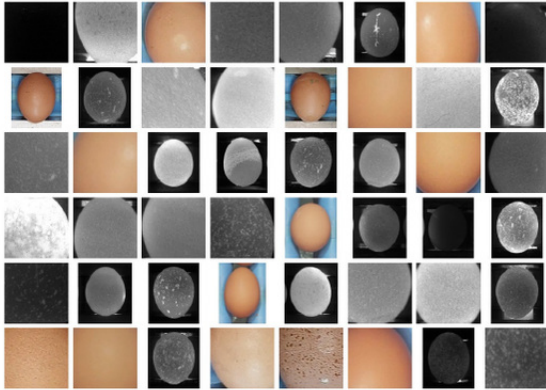
구분	항목	적용 내역	설명
S/W 개발환경	OS	Windows 개발 PC, Raspberry Pi OS 64-bit, Android/Web 테스트 환경	개발과 H/W 실행 환경 분리
S/W 개발환경	개발도구	Visual Studio Code, Flutter, Python venv, Git/GitHub	앱, 백엔드, AI 브리지 개발
S/W 개발환경	언어/프레임워크	Dart/Flutter, Python/Flask, SQL/MySQL	모바일 앱, API 서버, DB 구성
AI 개발환경	모델/라이브러리	Ultralytics YOLO, OpenCV, best.pt, yolov8n.pt	후보 탐지 및 식재료 분류
H/W 구성장비	디바이스	Raspberry Pi 5, 카메라 모듈, 리드스위치, 냉장고	촬영과 문 이벤트 감지
통신	연동 방식	HTTP REST, multipart upload, MJPEG preview stream	Pi, 서버, 앱 간 데이터 전달
관리환경	문서/형상관리	HANDOFF.md, RASPBERRY_PI.md, GitHub repository	실행 절차와 테스트 명령 표준화

| 참조-S/W 기능 실사 사진

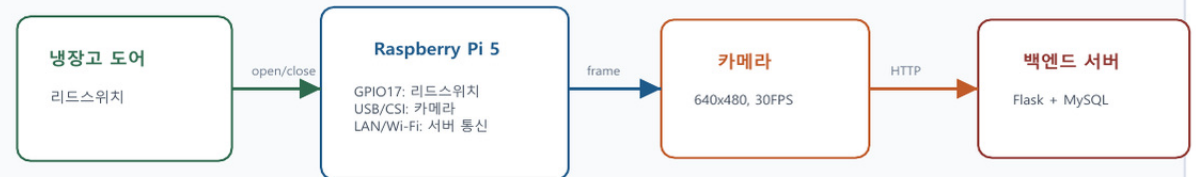


화면/기능	검증 내용	비고
재고 목록	검색, 필터, 이미지 카드, 확인 필요 상태 표시	Flutter 화면 구현
재고 상세	식재료명, 수량, 단위, 상태, 메모 수정	API update 연동
레시피 목록	보유 재료 기반 missing_count 정렬 및 부족 재료 표시	Flask recipes API 연동
업로드 반영	Raspberry Pi에서 /upload 호출 시 fridge_items에 저장	백엔드 구현 완료

| 참조-H/W 기능 실사 사진



식재료 이미지 데이터 및 분류 테스트 예시



기능	실사/검증 기준	확인 방법
카메라 촬영	냉장고 내부 프레임이 식재료를 충분히 포함해야 함	preview stream 또는 snapshot으로 확인
문 열림 감지	문 열림과 닫힘에서 GPIO 상태가 반전되어야 함	--trigger reed --once로 이벤트 확인
인식 박스 표시	후보 box와 label이 preview frame에 표시되어야 함	브라우저에서 http://PiIP:8080 확인
서버 업로드	원본 이미지와 crop 이미지가 uploads 폴더에 저장되어야 함	앱 재고 목록 및 DB row 확인

※ 현재 저장소에 실물 장착 사진이 없어 데이터셋 검증 이미지와 H/W 구성도로 대체함

| 참조-동영상 촬영 콘티

컷	화면	촬영 내용	자막/설명
1	문제 제시	냉장고 안 식재료를 기억하기 어려운 상황	독거노인과 1인 가구의 식재료 관리 부담
2	하드웨어 소개	Raspberry Pi, 카메라, 리드스위치 장착 위치	문 열림 이벤트 기반 자동 촬영
3	자동 인식	문 열림 후 preview stream에 box와 label 표시	YOLO/분류 모델로 식재료 인식
4	서버 저장	업로드 로그 또는 DB/백엔드 응답 확인	이미지, crop, label, confidence 저장
5	앱 재고 확인	Flutter 앱에서 재고 카드 갱신	사용자는 앱에서 재료와 수량 확인
6	오인식 수정	확인 필요 항목을 상세 화면에서 수정	AI 결과는 사용자가 쉽게 보완 가능
7	레시피 추천	보유 재료 기반 추천 목록과 부족 재료 확인	재고 데이터가 식사 준비로 연결
8	확장성	복지/돌봄 서비스 연계 가능성 설명	스마트홈 생활지원 서비스로 확장

| 참조-프로젝트 관리

구분	추진내용	4월	5월	6월	7월	8월	9월	10월
계획	주제 선정 및 목표 설정							
분석	문제 정의, 요구사항 분석							
설계	시스템/DB/API/UI/HW 구조 설계							
개발	Flask/MySQL 백엔드 및 재고 API							
개발	Flutter 앱 및 레시피 추천							
개발	AI 인식 및 Raspberry Pi 연동							
테스트	인식 정확도, H/W 장착 테스트							

관리 항목	내용
이슈관리	카메라 위치, 조명, 네트워크 주소, 인식률, 중복 업로드를 주요 이슈로 관리한다.
해결방안	preview stream, dry-run, once mode, stable frame, cooldown, 문 열림/닫힘 workflow를 통해 단계별 검증한다.
형상관리	GitHub repository와 HANDOFF.md, RASPBERRY_PI.md로 실행 절차와 작업 인수인계를 관리한다.
품질관리	API 응답, DB 저장, 앱 표시, H/W 이벤트를 각각 분리 테스트 후 통합 테스트한다.

Thank you

